

Java Code Analysis

Fifty-two solutions across three tiers, distilled into their libraries, algorithmic concepts and reusable tricks - with a code-review pass over pitfalls and a per-file index for revision.

Tier 1 - Tier 2 - Tier 3: 52 files, about 1,700 lines analysed

Contents

Section 1 - The Corpus at a Glance	3
How the corpus is organised	3
The one-paragraph summary	3
Section 2 - Libraries and APIs Used	4
The java.util collections	4
Sorting utilities: Arrays and Collections	5
The java.lang workhorses	5
I/O and testing utilities	6
APIs conspicuously absent	6
Section 3 - Concepts and Techniques Catalogue	6
Number base conversion (both directions)	6
Digit manipulation: the % 10 and / 10 pair	7
Number theory: primes, divisors and the sqrt(n) loop	8
Hashing: counting, grouping and membership	8
Sorting deep dive: four patterns built on Arrays.sort	9
Two pointers and sliding windows	10
Prefix sums and running aggregates	10
Bit manipulation	10
Strings and StringBuilder	11
In-place matrices and marker arrays	12
Linked lists via the LinkedList deque API	12
Recursion and permutations	12
Section 4 - Testing Patterns in the Corpus	13
Pattern 1: the assert-helper family	13
Pattern 2: expected-output tables	13
Pattern 3: stdout capture for exact-output methods	13
Pattern 4: demonstration mains	14
Section 5 - Tips and Tricks: The Master List	14
Characters, digits and small maths	14
Collections idioms	14

Sorting moves	15
Pointers, windows and prefixes	15
Bits and simulation	15
Strings and output	16
Structure, guards and exam discipline	16
Section 6 - Code Review Notes: Pitfalls Spotted and Fixes	16
The six findings that would fail test cases	17
Spec-ambiguity findings (verify against the problem statement)	17
Section 7 - Per-File Index	18
Tier 1 index (19 files)	18
Tier 2 index (21 files)	19
Tier 3 index (12 files)	20
Section 8 - Revision Checklist	21

SECTION

Section 1 - The Corpus at a Glance

This report analyses all 52 Java source files in the CodingQuestions repository - 19 Tier 1 solutions, 21 Tier 2 solutions and 12 Tier 3 solutions (about 1,700 lines of code). It records every library and API the code relies on, the algorithmic concepts and patterns it uses, the reusable tips and tricks worth carrying into the exam, and an honest code-review pass over the spots where the current versions have bugs or sharp edges.

How the corpus is organised

Each file solves one assessment-style coding problem. Every solution follows the same shape: a static helper method that does the work, and a main that exercises it - either by printing results or by running small self-written tests. The tiers mark increasing difficulty: Tier 1 is arithmetic, loops and strings; Tier 2 introduces hashing, sorting and two-pointer work; Tier 3 adds sliding windows, greedy simulation and matrix problems.

Tier	Files	Dominant themes	Typical complexity
Tier 1	19	Number bases, digit manipulation, primes and divisors, recursion, string basics	$O(n)$ or $O(\sqrt{n})$
Tier 2	21	HashMap/HashSet counting, sorting, anagrams, regex, StringBuilder building, greedy picks	$O(n \log n)$
Tier 3	12	Two pointers, sliding windows, in-place matrix marking, XOR simulation, linked lists	$O(n)$ to $O(n \log n)$

Corpus composition

The one-paragraph summary

If the whole corpus had to be compressed into one sentence: **almost every problem here is solved by counting something in a HashMap, sorting the data and then scanning it once, walking two pointers toward each other, or extracting digits with % and / -** and the answers are assembled with `StringBuilder` and guarded with small edge-case checks at the top of each method. The libraries are confined to `java.util` collections and `java.lang` workhorses (`Math`, `StringBuilder`, `Integer`, `String`); there is no external dependency anywhere.

The meta-pattern of this codebase. Count it, sort it, walk it, build it. A frequency map for counting, `Arrays.sort` for order-based insight, a single scan for the answer, and `StringBuilder` for output. Nine out of ten problems in this repository fall to one of those four moves.

SECTION

Section 2 - Libraries and APIs Used

The corpus deliberately sticks to the standard library. This section inventories every import and every heavily-used java.lang API, where it appears, and the specific idiom each file uses - the exact vocabulary an examiner (or a reviewer) sees when reading this code.

The java.util collections

Type	Used in	What it does there	Idiom to remember
HashMap	CountOccurrences, MostFrequentVowel, ReplaceMostFrequentCharacters, RhymeWords	Frequency counting; grouping words by suffix	<code>mp.put(k, mp.getOrDefault(k, 0) + 1)</code>
HashSet	RemoveDuplicates, IntersectionOfLists, LongestSubstring..., ReplaceMostFrequentCharacters	Deduplication; O(1) membership tests	<code>set.contains(x)</code> then <code>set.remove(x)</code> for one-time matching
Map.Entry	CountOccurrences, MostFrequentVowel, RhymeWords	Iterating key-value pairs	<code>for (Map.Entry<K,V> e : mp.entrySet())</code>
ArrayList / List	PalindromeInRange, IntersectionOfLists, MergeTwoSortedArrays, MatrixEvenOddSecondLargest, RhymeWords, PairSumwithMaximumProduct	Building result lists of dynamic length	<code>list.add(...); print via toString()</code>
LinkedList	LinkedLists, LinkedListPalindrome	Deque-style end operations	<code>getFirst() / getLast() / removeFirst() / removeLast()</code>

Collection types used across the 52 files

Two grouping idioms stand out. The first is the frequency counter, which appears in four different files and is the single most repeated pattern in the corpus:

```
HashMap<Character, Integer> mp = new HashMap<>();
for (char ch : str.toCharArray()) {
    mp.put(ch, mp.getOrDefault(ch, 0) + 1);
}
```

The second is the group-by-suffix pattern in RhymeWords, which uses `computeIfAbsent` with a lambda - the only lambda in the corpus, and the cleanest way to build a `Map<String, List<String>>` without a null check:

```
map.computeIfAbsent(suffix, key -> new ArrayList<>()).add(word);
```

Trick - the counting idiom. `getOrDefault(key, 0) + 1` replaces the three-line if-present-else dance. In an exam it saves time and removes the most common null-pointer mistake. Its sibling, `computeIfAbsent`, does the same favour for map-of-list grouping.

Sorting utilities: Arrays and Collections

Sorting appears in six files and comes in exactly two flavours: `Arrays.sort` for primitive arrays and char arrays (`IsAnagram`, `MaximumDifferenceBetweenSuccessiveElement`, `ChocolateDistribution`, `PairSumwithMaximumProduct`), and `Collections.sort` for `ArrayLists` (`MergeTwoSortedArrays`, `MatrixEvenOddSecondLargest`). None of the uses need a custom `Comparator` - every sort is natural ascending order, and the problems exploit the resulting order rather than the sorting itself. the sorting deep dive in Section 3 dissects each sort-dependent pattern.

One subtle API appears in the test code of `PalindromeInRange`: `Arrays.asList(1, 2, 3)` wraps literal values into a `List` in one call, which makes expected-value comparisons readable.

The java.lang workhorses

API	Where	Notes and tricks
<code>StringBuilder</code>	11 files - all string building	<code>append</code> , <code>insert(0, ...)</code> , <code>reverse()</code> , <code>delete(i, i+2)</code> , <code>charAt</code> , <code>length</code> . Never use <code>+=</code> on <code>Strings</code> in a loop.
<code>String</code>	everywhere	<code>split(" ")</code> , <code>trim()</code> , <code>substring</code> , <code>toCharArray()</code> , <code>charAt</code> , <code>matches(regex)</code> , <code>format</code> , <code>repeat</code> (Java 11+), <code>indexOf</code> as membership test.
<code>Math</code>	8 files	<code>abs</code> , <code>max</code> , <code>min</code> , <code>sqrt</code> , <code>pow</code> . <code>Math.abs(arr[i] - num)</code> for distance checks; <code>Math.max</code> inside scans.
<code>Integer</code>	<code>RearrangementOfBits</code> , <code>ChocolateDistribution</code>	<code>Integer.bitCount(n)</code> counts set bits; <code>Integer.MAX_VALUE</code> as a safe 'infinity' for min-tracking.
long arithmetic	<code>FibonacciSeries</code>	Fibonacci terms overflow <code>int</code> past term 46 - the code uses <code>long</code> for the running terms.

java.lang APIs the corpus depends on (no import needed)

I/O and testing utilities

The only `java.io` usage in the corpus is a testing trick, not real I/O. `FibonacciSeries` and `StandardDeviation` capture their own stdout so the exact printed output can be compared against an expected string:

```
PrintStream originalOut = System.out;
ByteArrayOutputStream output = new ByteArrayOutputStream();
try {
    System.setOut(new PrintStream(output));
    printFib(n);           // method under test
} finally {
    System.setOut(originalOut); // always restore
}
```

This matters for the assessment directly: coding rounds diff the **exact** stdout, so being able to assert on captured output locally - including newlines, via the `replace(System.lineSeparator(), "\n")` normalisation in `StandardDeviation` - is the cheapest way to catch formatting bugs before submitting.

Exam trap this trick defuses. Wrong output format is the silent killer: a trailing space, a missing newline or a prompt line like `Enter a number:` fails otherwise-correct solutions. Capturing stdout and diffing it against the expected block catches all three before submission.

APIs conspicuously absent

It is just as informative to note what the corpus never uses: no `Scanner` or `BufferedReader` (inputs are hardcoded test arrays, since the round grades exact output), no Java Streams, no third-party libraries, no custom `Comparator` or lambda-based sorting, and no `Arrays.asList`-backed algorithms beyond test data. Everything is deliberately hand-rolled so the algorithm itself is visible - which is exactly the skill the coding round grades.

SECTION

Section 3 - Concepts and Techniques Catalogue

This section walks through every algorithmic concept the corpus uses, from base conversion to sliding windows. Each concept names the files that use it, shows the core idiom as it actually appears in the code, and states the complexity the technique achieves - so it can double as a pattern-revision sheet.

Number base conversion (both directions)

Three files convert between bases, and each uses a different half of the standard toolkit. `BinaryToDecimal` consumes a binary string left to right with the positional accumulator - multiply the running value by the base, add the next digit. `DecimalToBinary` and `SumOfBinaryDistance` go the other way: repeatedly divide by 2, collect remainders, and reverse at the end.

```
// binary -> decimal (left to right)
decimal = decimal * 2 + (binary.charAt(i) - '0');

// decimal -> binary (right to left, then reverse)
while (decimal > 0) {
    rsl.append(decimal % 2);
    decimal /= 2;
}
return rsl.reverse().toString();
```

Trick - char to digit. `binary.charAt(i) - '0'` converts the character '1' to the integer 1 by subtracting code points. The inverse trick appears in `StringDecoder`: `(char) ('A' + len - 1)` maps a run length to a letter. Both avoid any parsing library.

`DecimalToBinary` also shows the zero guard: `if (decimal == 0) return "0";` before the loop, because a `while (n > 0)` loop would otherwise return an empty string. Every `while (n > 0)` digit loop in the corpus carries this same implicit edge case.

Digit manipulation: the % 10 and / 10 pair

Six files extract digits without ever converting to a string: `n % 10` peels the last digit, `n /= 10` drops it. `GooglyPrimeNumbers` sums digits, `PalindromeInRange` builds the reversed number (`reverse = reverse * 10 + digit`), `CountCarry` simulates column addition digit by digit, `EncodeCharacter` squares each digit, and `MaxExponentOf2` counts how many times a number can be halved - which is `floor(log2 n)`, the exponent of the largest power of 2 dividing the search range.

```
// CountCarry: simulate adding two numbers column by column
while (num1 != 0 || num2 != 0) {
    int sum = currentCarry + num1 % 10 + num2 % 10;
    if (sum > 9) { currentCarry = 1; carryCount++; }
    else currentCarry = 0;
    num1 /= 10; num2 /= 10;
}
```

`CountCarry` shows the digit-loop refinement worth remembering: the loop condition is `num1 != 0 || num2 != 0` (either), not and - otherwise unequal-length numbers stop early. And the carry is *propagated* into the next column before being reset, which is the whole simulation.

Trick - max exponent of 2 without logs. `countTwos(n)` divides by 2 until zero and counts the divisions: no `Math.log`, no floating point, no off-by-one. It is the integer-only way to get `floor(log2 n)` and it is safe for every positive int.

Number theory: primes, divisors and the sqrt(n) loop

Three files do number theory, and all three use the same optimisation: loop only to the square root, expressed overflow-safely as `i * i <= n` rather than `i <= Math.sqrt(n)`. `GooglyPrimeNumbers` and `SumOfPrimesLessThanN` use it for primality tests; `SumOfDivisors` extends it to the divisor-pair method - when `i` divides `n`, both `i` and `n / i` are divisors, so add both and guard against counting the square root twice with `i != n / i`.

```
for (int i = 1; i <= Math.sqrt(n); i++) {
    if (n % i == 0) {
        sum += i;
        if (i != n / i) sum += n / i; // avoid double-count
    }
}
```

Two smaller number-theory moves: `SumOfNumbersDivisibleByBoth3And5` collapses 'divisible by 3 and 5' into a single `i % 15 == 0` check (15 is the LCM), and `MissingNumber` uses the Gauss formula `n * (n + 1) / 2` so the missing value is one subtraction instead of a search.

Trick - LCM collapse. Whenever a condition says 'divisible by A and B', check divisibility by `LCM(A, B)` instead. One modulo instead of two, and it reads as the mathematical statement it is.

Hashing: counting, grouping and membership

The `HashMap` counting idiom was covered in Section 2; the *decisions* around it are what make it a technique. Three distinct uses appear:

- **Frequency counting** (`CountOccurrences`, `MostFrequentVowel`, `ReplaceMostFrequentCharacters`) - build the count map, then scan entries for the answer. The scan compares `entry.getValue()` against a running max and pulls `entry.getKey()` as the answer.
- **Grouping** (`RhymeWords`) - key by derived property (the last `rhymeLength` characters, lower-cased), value is the list of matching words; groups with `size() >= 2` are the rhyme families.
- **Membership and dedup** (`RemoveDuplicates`, `IntersectionOfLists`, `LongestSubstringWithoutRepeatingCharacters`) - a `HashSet` answers 'seen it?' in $O(1)$. `IntersectionOfLists` adds one refinement: after matching, it removes the element so each set entry matches at most once.

Trick - indexOf as a membership test. `"aeiouAEIOU".indexOf(ch) != -1` is the corpus's vowel test: no Set, no loop, one line. For a fixed tiny alphabet it beats a `HashSet` on clarity, and the same string doubles for both cases at once.

One caveat the corpus itself demonstrates: `HashMap` iteration order is unspecified. `MostFrequentVowel` breaks ties with 'first max wins' during `entrySet()` iteration, so which vowel it returns on a tie depends on the map's internal hashing. If deterministic output matters, use a `TreeMap` (sorted keys) or `LinkedHashMap` (insertion order), or add an explicit tie-break rule.

Sorting deep dive: four patterns built on `Arrays.sort`

Sorting is the highest-leverage library call in the corpus: five problems become almost trivial once the data is ordered. Four distinct sort-then-act patterns appear:

Pattern 1 - sort to compare canonically (anagram check)

`IsAnagram` sorts both strings' char arrays; two words are anagrams exactly when the sorted forms are equal. The cheap length guard up front (`s.length() != t.length()`) skips all work for the obvious mismatch. Cost: $O(n \log n)$ against the $O(n)$ counting alternative - chosen here because the sorted comparison is three lines and hard to get wrong.

Pattern 2 - sort then scan neighbours (minimum adjacent gap)

`MaximumDifferenceBetweenSuccessiveElement` answers 'maximum difference between successive elements' by sorting first: after sorting, the answer is the largest gap between *adjacent* sorted values, a single $O(n)$ scan. This is the classic realisation that 'successive' means 'neighbouring in sorted order', not 'consecutive in the input'.

```
Arrays.sort(arr);
int maxDiff = 0;
for (int i = 0; i < arr.length - 1; i++) {
    maxDiff = Math.max(maxDiff, Math.abs(arr[i+1] - arr[i]));
}
```

Pattern 3 - sort then slide a window (chocolate distribution)

`ChocolateDistribution` sorts the packets, then every window of m consecutive sorted values is a candidate group; the answer is the minimum of `arr[i + m - 1] - arr[i]` across all windows. Sorting guarantees each window is the tightest possible group containing its values, so no other combination can beat it. The loop runs `i <= arr.length - m` - the window arithmetic that keeps the last full window in range.

Pattern 4 - sort then converge two pointers (pair sum)

`PairSumwithMaximumProduct` sorts, then walks `start` and `end` inward comparing `arr[start] + arr[end]` with the target, tracking the maximum product among matches. Section 3.5 covers the two-pointer mechanics - the sorting here is what makes the sum comparison actionable (move the pointer that moves the sum toward the target).

Trick - sort changes what is adjacent. Three of these four patterns share one idea: after sorting, 'next to each other' becomes meaningful. Gaps, windows and pairs all become local scans. If a problem mentions closest, smallest-range, or most-balanced grouping, try `Arrays.sort` first and re-read the problem in sorted space.

`MatrixEvenOddSecondLargest` adds a variant: `Collections.sort` on two `ArrayLists` after splitting by index parity, then reading the second largest directly as `index.size() - 2`. Sorting turns 'second largest' into a subscript - no second-scan bookkeeping needed. (The single-scan alternative tracks `max` and `secondMax` in one pass; see the checklist in Section 6.)

Two pointers and sliding windows

Four pointer arrangements appear, increasing in sophistication:

Pattern	Files	Mechanics	Complexity
Converging pointers	PairSumwithMaximumProduct	start/end walk inward; move the side that moves the sum toward the target	$O(n \log n)$ with sort
Both-ends swap	SumatEvenIndicesoftheReversedArray	swap <code>arr[left]</code> and <code>arr[right]</code> , step inward - in-place array reversal	$O(n)$ time, $O(1)$ space
Ends-only access	LinkedListPalindrome	compare <code>getFirst/getLast</code> , then <code>removeFirst/removeLast</code> - shrink from both ends	$O(n)$
Variable sliding window	LongestSubstringWithoutRepeatingCharacters	right expands, left shrinks on duplicate via <code>HashSet</code>	$O(n)$

Pointer patterns in the corpus

`LongestSubstringWithoutRepeatingCharacters` is the corpus's flagship window: right scans forward, and while the new character is already in the set, the left edge removes characters and advances. The window `[left..right]` is therefore always duplicate-free, and `maxLen = Math.max(maxLen, right - left + 1)` records the best span. `MaxFavouriteSong` is the fixed-width sibling: every substring window of width `k` is measured (brute-force $O(n \cdot k)$; a running count of 'a' characters entering/leaving the window would make it $O(n)$).

Trick - reverse by mapping, not by mutating. `SumatEvenIndicesoftheReversedArray` reverses in place, then sums even indices. The insight worth keeping: reversing maps index `i` to `n - 1 - i`, so the answer could be computed directly from the original array - sum every `arr[i]` where `(n - 1 - i) % 2 == 0`. Same result, no mutation, no second pass.

Prefix sums and running aggregates

`EquilibriumSum` is the corpus's one prefix-sum showcase. Compute the total once; then for each index, `rightSum = totalSum - leftSum - arr[i]` and compare with `leftSum` - no nested loops, $O(n)$ time, $O(1)$ space, with `-1` as the not-found sentinel. `NegativeStockPriceDays` uses the same adjacent-scan discipline for a simpler question (count `arr[i+1] < arr[i]`), and `StandardDeviation` accumulates mean and mean-squared-deviation in two linear passes - notably in integer arithmetic, which keeps the printed output exact but truncates decimals.

Bit manipulation

Two files use bits deliberately, and both pay off handsomely:

- `Integer.bitCount(n)` (`RearrangementOfBits`) - the population count, straight from the standard library. The problem 'rearrange all 1-bits to the front' reduces to: count the ones, then rebuild $2^k - 1$ by `result = (result << 1) | 1`, `k` times.

- **XOR as a state flip** (BulbSwitch) - the greedy bulb problem in $O(n)$. `current = arr[i] ^ flipped` applies the accumulated flip state without touching the prefix; when a bulb is off, increment the switch count and toggle `flipped ^= 1`. The naive version re-flips every remaining bulb per switch and costs $O(n^2)$.

```
// BulbSwitch: XOR carries the "has this index been flipped an odd number of times" state
int current = arr[i] ^ flipped;
if (current == 0) { count++; flipped ^= 1; }
```

Trick - XOR is a toggle. $x \oplus 1$ flips a bit, $a \oplus b$ differs two values, and XOR-ing the same state twice cancels. Whenever a simulation says 'flip everything after position *i*', store the *parity* of flips instead of doing the flips - that is the $O(n^2)$ to $O(n)$ jump.

Strings and StringBuilder

String problems dominate Tier 2, and every one of them assembles its answer with `StringBuilder` rather than String concatenation (concatenating in a loop copies the string each time - $O(n^2)$ total). Three building styles appear:

- **Append then reverse** (DecimalToBinary, ReverseWords) - build in the natural order, call `reverse()`. Best when later elements belong earlier (remainders, word order).
- **Insert at front** (EncodeCharacter) - `insert(0, ...)` builds left-to-right for digit streams. Correct but each insert shifts the buffer; for exam-sized inputs it is fine, append-plus-reverse is the general habit.
- **Rebuild with a filter** (MoveHyphenToFront, ReplaceMostFrequentCharacters) - count what moves or changes, then append characters conditionally. `"-".repeat(hCount)` (Java 11+) seeds the hyphen block in one call.

Four string-specific tricks round out the catalogue: `String.split(" ")` tokenises into a word array for `FirstKWords`, `LongestWord` and `ReverseWords`; `trim()` normalises edge spaces before splitting or scanning (`LengthOfLastWord`, `ReverseWords`); `substring(i + 1)` after finding a delimiter extracts the tail; and `String.matches(regex)` validates a whole string in one call - `PasswordChecker` packs three requirements into one look-ahead pattern, with a null check short-circuiting before the regex runs.

The adjacent-duplicate stack trick

`RemoveAdj` removes pairs of adjacent equal characters (abbaca becomes ca) with a `StringBuilder` used like a stack: when `charAt(i) == charAt(i + 1)`, `delete(i, i + 2)` both characters, then rewind the index with `i = Math.max(0, i - 1)` so the newly-adjacent pair left behind is re-checked. No explicit stack, one buffer, $O(n)$ amortised.

```
if (builder.charAt(i) == builder.charAt(i + 1)) {
    builder.delete(i, i + 2);
    i = Math.max(0, i - 1); // re-check the seam
} else { i++; }
```

Run-length decoding

StringDecoder splits the binary string on '0'; each remaining token is a run of 1s whose length maps to a letter via `(char) ('A' + length - 1)`. Split-on-delimiter plus a length-to-value mapping solves in four lines what looks like a parser problem.

In-place matrices and marker arrays

SetMatrixZeros is the corpus's 2-D showcase: one pass records which rows and columns contain a zero into `boolean[] rowZeros` and `boolean[] colZeros`; a second pass rewrites cells whose row or column is marked. This is the $O(m + n)$ extra-space variant of the classic problem - clear and safe. The famous $O(1)$ -space version instead uses the matrix's own first row and column as the markers, at the cost of fiddly bookkeeping; for an exam the marker arrays are the version you can write without bugs.

MatrixEvenOddSecondLargest also traverses a 2-D mindset on 1-D data - partition by index parity (`i % 2 == 0`), sort each partition, and read answers by subscript.

Linked lists via the LinkedList deque API

LinkedListPalindrome never touches nodes or pointers: it uses `java.util.LinkedList`'s double-ended API - compare `getFirst()` with `getLast()`, then `removeFirst()` and `removeLast()`, until fewer than two elements remain. It is the two-pointer palindrome technique expressed through a library type. The trade-off to know: `get(i)` on a `LinkedList` is $O(n)$ (it walks from the nearest end), which is why this code uses only the end operations - each is $O(1)$.

Trick - LinkedList ends over indexing. When a `LinkedList` is the given structure, reach for `getFirst/getLast/removeFirst/removeLast` and never index into it. Every indexed access on a linked list hides a linear walk; the end operations are genuinely constant time.

Recursion and permutations

Two files recurse. `NthFibonacciTerm` is the textbook doubly-recursive definition with a base case at `n <= 2` (and it deliberately folds `n = 0` and negatives into that base case) - correct but $O(2^n)$; the iterative two-variable version in `FibonacciSeries` is the same answer in $O(n)$ time and $O(1)$ space, and is the version to write when `n` can be large. `VowelPermutation` recurses for factorial: count the consonants, and the number of ways to arrange them is `consonantCount!` - a counting insight (`n * fact(n - 1)`) rather than an iteration problem.

The pairing teaches the exam lesson directly: recursion states the recurrence beautifully; iteration executes it cheaply. Recognise the recurrence (factorial, Fibonacci, digit sums) from the base case, then choose the loop when size demands it.

SECTION

Section 4 - Testing Patterns in the Corpus

Roughly half the files test themselves inside main with hand-rolled assertion helpers. This section records the four testing patterns the corpus uses, because they are directly reusable in an exam environment where no JUnit is available and output must be exact.

Pattern 1: the assert-helper family

GooglyPrimeNumbers, SumOfPrimesLessThanN, SumOfDivisors, DecimalToBinary, NthFibonacciTerm, CountCarry, AbsoluteDifference and StringDecoder all define tiny private helpers - assertEquals, assertTrue, assertFalse - that throw AssertionError with a message naming the case. The pattern is three lines per helper and turns main into a readable test suite:

```
private static void assertEquals(int expected, int actual) {
    if (expected != actual) {
        throw new AssertionError("Expected " + expected + ", but got " + actual);
    }
}
```

The same files show the discipline that makes the helpers valuable: tests cover **zero** (digitSum(0), findPrimeSum(0)), **one** (isPrime(1) is false), **negatives** (nthFib(-1)), **empty input** (absDiff on {} returns -1), and **boundaries** (n == 1 in SumOfDivisors returns 0, since 1 is not a proper divisor of itself).

Pattern 2: expected-output tables

StringDecoder and PalindromeInRange store test cases as data - a String[][] of {input, expected} pairs, or Arrays.asList(...) expectations - and loop over them. Adding a case is one array row, not a new block of code. This is the lightest-weight table-driven test possible in plain Java.

```
String[][] testCases = { { "10110111", "ABC" }, { "1", "A" }, { "11110111", "DC" } };
for (String[] testCase : testCases) {
    if (!decodeString(testCase[0]).equals(testCase[1]))
        throw new AssertionError(...);
}
```

Pattern 3: stdout capture for exact-output methods

Methods that print instead of returning (FibonacciSeries.printFib, StandardDeviation.deviation) are tested by swapping System.out for a ByteArrayOutputStream-backed PrintStream, restoring it in a finally, and comparing the captured string against the exact expected block. StandardDeviation adds the newline normalisation (replace(System.lineSeparator(), "\n")) so the test is OS-independent. This is the pattern that guards the one thing an online judge actually grades.

Pattern 4: demonstration mains

The remaining files (Table, LinkedLists, MaxInArray, ReplaceCharacter, MoveHyphenToFront) print worked examples across several inputs. Useful while developing, but weaker than the assert patterns - nothing fails if an output changes. The corpus trend is clear though: earlier files demonstrate, later files assert.

SECTION

Section 5 - Tips and Tricks: The Master List

Every reusable technique observed in the 52 files, condensed into one checklist. The arrow points at the concrete trick; the text says when to reach for it. If you revise only one section of this report before the coding round, make it this one.

Characters, digits and small maths

1. `charAt(i)` - `'0'` converts a digit character to its integer value; `(char) ('A' + k - 1)` maps 1..26 to letters. No parsing libraries needed.
2. `n % 10` peels the last digit, `n /= 10` drops it; loop while `(n > 0)` for digits, but guard the `n == 0` case separately (DecimalToBinary does).
3. Loop while `(num1 != 0 || num2 != 0)` - or, not *and* - when two numbers' digits are consumed in lockstep (CountCarry).
4. `i * i <= n` is the overflow-safe, allocation-free way to loop to `sqrt(n)` for primes and divisors; add the `i != n / i` guard when collecting divisor pairs.
5. Divisible by A and B? Check `x % LCM(A,B) == 0` - one modulo (the % 15 trick).
6. Missing number in 1..n? Gauss: `n * (n + 1) / 2 - actualSum` - no array scan for the gap.
7. `floor(log2 n)` without logs: count how many times `n /= 2` until zero (MaxExponentOf2).
8. `Math.abs`, `Math.max`, `Math.min` replace four-line if-blocks; `Integer.MAX_VALUE` is the safe starting point for min-tracking.
9. Watch overflow: Fibonacci passed term 46 overflows `int` - FibonacciSeries uses `long`.
10. Numerically fragile formula? Rearrange it. RootsOfQuadraticEquation computes roots via `q = -0.5 * (b + sign(b) * sqrt(disc))`, `root1 = q / a`, `root2 = c / q` to avoid catastrophic cancellation - and handles `a == 0` (linear) and `b == 0 && c == 0` (infinite solutions) before touching the discriminant.

Collections idioms

1. Frequency counting in one line: `mp.put(k, mp.getOrDefault(k, 0) + 1)`.
2. Grouping into a map of lists in one line: `map.computeIfAbsent(key, x -> new ArrayList<>()).add(item)`.
3. HashSet for 'have I seen it?', plus `set.remove(x)` when an element may match only once (IntersectionOfLists).
4. Iterate with `for (Map.Entry<K, V> e : map.entrySet())` when you need key *and* value.

5. HashMap iteration order is unspecified - if a tie-break matters, use TreeMap/LinkedHashMap or compare explicitly (MostFrequentVowel returns hash-order-dependent answers on ties).
6. LinkedList: only getFirst/getLast/removeFirst/removeLast - never index into it; get(i) is O(n).
7. Arrays.sort for arrays and char arrays, Collections.sort for Lists; expected values as Arrays.asList(...) make test data one-liners.
8. "-".repeat(count) (Java 11+) seeds a repeated block without a loop (MoveHyphenstoFront).

Sorting moves

1. Anagram check: length guard, then sort both char arrays and compare (IsAnagram).
2. 'Max/min difference between successive elements' means *sorted-order neighbours* - sort, then scan `arr[i+1] - arr[i]` (MaximumDifferenceBetweenSuccessiveElement).
3. Smallest range covering m items: sort, then slide a window of width m and minimise `arr[i + m - 1] - arr[i]` (ChocolateDistribution).
4. Pair with a target sum: sort, converge two pointers; move start up when the sum is small, end down when it is large (PairSumwithMaximumProduct).
5. Second largest: sort and read index `size() - 2` - or track max and secondMax in one scan if sorting is too expensive (MatrixEvenOddSecondLargest).
6. Sorting is O(n log n) - if the problem only needs counts or extrema, a single scan or a counting array is cheaper. Sort when *order* answers the question.

Pointers, windows and prefixes

1. In-place reversal: swap `arr[left]` and `arr[right]`, step both inward, stop at `left >= right` (SumatEvenIndicesoftheReversedArray).
2. Prefix sums kill nested loops: `rightSum = total - leftSum - arr[i]` in one pass (EquilibriumSum); return -1 as the sentinel.
3. Variable sliding window: expand right, shrink left while a HashSet still contains the incoming character; track `right - left + 1` (LongestSubstringWithoutRepeatingCharacters).
4. Fixed window: instead of recounting each window, keep a running count - add the entering character, drop the leaving one (MaxFavouriteSong is the brute-force version; O(n·k) becomes O(n)).
5. Adjacent-pair scans use `arr.length - 1` as the bound - off-by-one lives exactly there (NegativeStockPriceDays).

Bits and simulation

1. Integer.bitCount(n) counts set bits; rebuild all-ones from them with `result = (result << 1) | 1` (RearrangementOfBits).
2. XOR is a toggle: `x ^ 1` flips, applying twice cancels. Carry flip *state* in one int instead of re-flipping a whole prefix (BulbSwitch: O(n²) becomes O(n)).
3. Greedy simulations usually need only local state - the current element plus one carried flag. Find that flag before writing loops.

Strings and output

1. Never += Strings in a loop; use `StringBuilder.append` (eleven files follow this; `FirstKWords` shows the concat habit to avoid).
2. Build-then-reverse beats insert-at-front for anything built right-to-left (remainders, reverse word order).
3. Mutually swapping two characters? Use `if / else if` in the same pass, or the second replacement overwrites the first (`ReplaceCharacter`).
4. Adjacent-pair removal: `StringBuilder.delete(i, i + 2)` then rewind `i = Math.max(0, i - 1)` to re-check the seam (`RemoveAdj` - a stack without a stack).
5. Run-length decode via `split` on the delimiter, then map run length to value (`StringDecoder`).
6. One regex with look-aheads validates multi-rule strings: `(?=.[0-9])(?=.[A-Z])...` - and null-check before calling matches (`PasswordChecker`).
7. Membership in a tiny alphabet: `"aeiouAEIOU".indexOf(ch) != -1` - one line, no Set.
8. `split(" ") + trim()` handles stray spaces; but note `split` drops trailing empty tokens - trim first.

Structure, guards and exam discipline

1. Guard the edges before the loop: zero, one element, empty input, null, negatives. The corpus's correct answers almost always start with a two-line early return.
2. Sentinel returns are part of the spec: `count > 0 ? count : -1` (`AbsoluteDifference`), return `-1` for not-found (`EquilibriumSum`). Read the Notes block for the exact sentinel the judge expects.
3. Seed extrema from `arr[0]`, not `Integer.MIN_VALUE`, when the array can be all-negative and non-empty (`MaxInArray`).
4. Print exactly what the problem states: no prompts, no trailing whitespace. The stdout-capture test pattern (Section 4) verifies this mechanically.
5. Write the tiny assert helpers first, then the solution - the tests document the intended edge cases while the algorithm is fresh.
6. Keep helpers static and stateless; pass everything in as parameters - every file here follows that shape, which makes methods trivially testable.

SECTION

Section 6 - Code Review Notes: Pitfalls Spotted and Fixes

An honest review pass over the corpus. None of these are stylistic nitpicks for their own sake - each one is a bug or a sharp edge that would fail a test case, and each fix is a technique worth absorbing. Reading other people's bugs (including your own past ones) is the fastest way to stop writing them.

The six findings that would fail test cases

1. FirstKWords - wrong loop bound. The loop condition is `i < k && i < str.length()`, but `i` indexes words, so the second guard must be `words.length` (character count vs word count). For inputs where the word count exceeds the character count the current guard never protects against `ArrayIndexOutOfBoundsException`. Also, `append(words[i] + " ")` concatenates a `String` inside the loop - the exact habit the rest of the corpus avoids; use `append(words[i]).append(' ')`.

2. MergeTwoSortedArrays - wrong length used for the second array. The second copy loop runs for `(int i = 0; i < arr1.length; i++) { list.add(arr2[i]); }` - `arr2` is indexed by `arr1`'s length, which only works because the demo inputs happen to be equal-length (and `newArr` is allocated but never used). Fix the bound to `arr2.length`. Also note the approach itself: concatenating and sorting is $O((n+m) \log(n+m))$; the true merge walk of two already-sorted arrays is $O(n+m)$ - worth knowing both.

3. LongestSubstringWithoutRepeatingCharacters - returns the debug value. The method returns `set.toString()` (the current window's characters) instead of the window length or the best substring. The algorithm is right; the return line is a leftover. On a judge, that scores zero for a correct solution - always re-read the method's contract before submitting.

4. PairSumwithMaximumProduct - inverted pointer movement. After sorting, when `currentSum < target` the code decrements `end`, which makes the sum *smaller* - moving away from the target forever. The standard walk moves `start` up when the sum is too small and `end` down when it is too large. As written, the demo input `{11, 1, 2, 8, 10, 11, 15, 7}` with target 18 produces an empty result and misses the answer pair `{8, 10}` (product 80):

```
if (currentSum == target) { ... start++; end--; }
else if (currentSum < target) { end--; } // WRONG: sum shrinks further
else { start++; } // WRONG: sum grows further
// Fix: < target -> start++; > target -> end--;
```

5. ChocolateDistribution - shortcut reads the unsorted array. The `arr.length == m` early return computes `Math.abs(arr[0] - arr[m - 1])` *before* `Arrays.sort` runs. When the whole array is one group, the answer is `max - min`, which equals that expression only if the input was already sorted. Move the sort above the guard, and the shortcut becomes correct for free.

6. MatrixEvenOddSecondLargest - no guard for short partitions. Reading `index size() - 2` throws `IndexOutOfBoundsException` when a parity partition has fewer than two elements (for example a one-element array has an empty odd partition). The fix is the corpus's own habit elsewhere: guard the small case first. Minor sibling: `.intValue()` after `get()` is redundant - auto-unboxing already yields an `int`.

Spec-ambiguity findings (verify against the problem statement)

- **LinkedListPalindrome** returns `false` for a size-1 list (`size() < 2`). A single element is conventionally a palindrome; if the judge says so, change the guard to `size() < 1` - or to an empty-list special case only.
- **LengthOfLastWord** returns `-1` when the input has no space (a single word). The usual contract is that a lone word's last word is itself, so the length should be returned. Side note: the method prints the substring as it goes - printing inside a worker method mixes responsibilities.

- **RemoveDuplicates** prints through a `HashSet`, so output order is hash order, not first-seen order. If the judge expects insertion order, `LinkedHashSet` preserves it with the same one-line API. The file's unused `HashMap/Map` imports are leftovers.
- **MostFrequentVowel** and **ReplaceMostFrequentCharacters** break frequency ties by 'first max seen while iterating the `HashMap`' - hash order, hence unspecified. Add an explicit tie rule (smallest character wins) if output must be deterministic.
- **ReverseWords** declares `static void main` without `public` - some JDK launcher versions will not run it. `public static void main` costs nothing and runs everywhere.
- **StandardDeviation** does integer arithmetic throughout (mean and mean-square both divide with truncation). Fine when the expected output is integer-formatted, but the name says deviation - know that this prints MSD, not the true standard deviation.
- **EncodeCharacter** builds its answer with `insert(0, ...)` - $O(n)$ per insert, $O(n^2)$ overall. Append-then-reverse is the linear habit.
- **SumOfPrimesLessThanN** starts its candidate loop at 1; `isPrime(1)` is guaranteed false, so start at 2. Harmless, but it is the kind of half-wasted iteration worth noticing.

The review lesson. Every finding above is a *contract* failure, not an algorithm failure: a wrong loop bound, a leftover return, an inverted comparison, an unguarded index. The algorithms were all correct. In a judged round, re-reading your own method's signature, bounds and return statement is worth more than one more optimisation.

SECTION

Section 7 - Per-File Index

The complete map of the corpus: every file, the concept it exercises, its key trick or API, and its complexity. Use it as a spaced-repetition index - pick a row, rewrite the solution from memory, then check the trick column.

Tier 1 index (19 files)

File	Concept	Key trick / API	Complexity
BinaryToDecimal	Base conversion	Positional accumulator; <code>charAt(i) - '0'</code>	$O(n)$
CountOccurrences	Frequency counting	<code>HashMap</code> + <code>getOrDefault</code> ; <code>entrySet</code> scan	$O(n)$
DecimalToBinary	Base conversion	$\% 2$ + append + reverse; zero guard	$O(\log n)$
FibonacciSeries	Iterative sequence	Two rolling variables; long; stdout-capture tests	$O(n)$
FirstKWords	String tokens	<code>split</code> + <code>StringBuilder</code> (bound bug noted in Section 6)	$O(n)$
GooglyPrimeNumbers	Primes + digits	$i * i \leq n$ test; digit-sum helper	$O(\sqrt{n})$

File	Concept	Key trick / API	Complexity
LengthOfLastWord	String scan	trim + reverse scan + substring	O(n)
MaxInArray	Single-pass max	Seed from arr[0]; track index of max	O(n)
MissingNumber	Closed-form sum	Gauss $n(n+1)/2$ minus actual	O(n)
NthFibonacciTerm	Recursion	Base case $n \leq 2$ (exponential - iterate instead)	O(2^n)
PalindromeInRange	Palindrome numbers	Reverse via % 10 / * 10; Arrays.asList tests	O(range)
RemoveDuplicates	Deduplication	HashSet (order caveat in Section 6)	O(n)
RootsOfQuadraticEquation	Algebra	Discriminant branches; cancellation-safe form	O(1)
StandardDeviation	Statistics	Integer mean/MSD; stdout-capture tests	O(n)
SumOfDistancesBetweenThreePoints	Geometry	sqrt/pow helper reuse	O(1)
SumOfDivisors	Number theory	Divisor pairs to sqrt(n); $i \neq n/i$ guard	O(sqrt n)
SumOfNumbersDivisibleByBoth3And5	Divisibility	LCM collapse: % 15	O(n)
SumOfPrimesLessThanN	Primes	Trial division per candidate	O(n sqrt n)
Table	Loops	Fixed 10-iteration table + running sum	O(1)

Tier 1 - fundamentals

Tier 2 index (21 files)

File	Concept	Key trick / API	Complexity
AbsoluteDifference	Filter count	Math.abs window; -1 sentinel	O(n)
CountCarry	Digit simulation	Column-add carry propagation; loop bound	O(digits)
EncodeCharacter	Digit mapping	Squares; insert(0) building (see Section 6)	O(n^2)
EquilibriumSum	Prefix sums	rightSum = total - left - cur; -1 sentinel	O(n)
IntersectionOfLists	Set operations	HashSet contains + remove-on-match	O(n + m)
IsAnagram	Canonical form	Sort char arrays; length guard first	O(n log n)
LongestWord	String scan	Max length + index over split words	O(n)
MaxExponentOf2	Integer log2	Halving count; tie-break to smaller value	O(range log n)

File	Concept	Key trick / API	Complexity
MaximumDifferenceBetweenSuccessiveElement	Sort + scan	Adjacent gap after sort	$O(n \log n)$
MergeTwoSortedArrays	Sorting	Concat + Collections.sort (bound bug in Section 6)	$O((n+m) \log)$
MostFrequentVowel	Hashing	Count map; indexOf vowel test (tie caveat)	$O(n)$
MoveHyphenToFront	String rebuild	count + "-".repeat + filtered append	$O(n)$
NegativeStockPriceDays	Adjacent scan	Count $arr[i+1] < arr[i]$; length - 1 bound	$O(n)$
PasswordChecker	Regex validation	Look-ahead pattern; null short-circuit	$O(n)$
RearrangementOfBits	Bit manipulation	Integer.bitCount; $(result \ll 1) 1$	$O(\text{bits})$
ReplaceCharacter	In-place string	Mutual swap via if / else if on char[]	$O(n)$
ReplaceMostFrequentCharacters	Hashing + greedy	k rounds of max-frequency pick	$O(n + k \cdot m)$
ReverseWords	Tokenising	trim + split + reverse-order append	$O(n)$
RhymeWords	Grouping	computeIfAbsent by suffix; toLowerCase	$O(n \cdot L)$
SumatEvenIndicesoftheReversedArray	Two pointers	In-place swap reversal; index mapping insight	$O(n)$
SumOfBinaryDistance	Base conversion	Remainder building; $0 \rightarrow 1$ / $1 \rightarrow 2$ positional remap	$O(\log n)$

Tier 2 - collections, sorting and strings

Tier 3 index (12 files)

File	Concept	Key trick / API	Complexity
BulbSwitch	Greedy simulation	XOR carries flip state - $O(n)$ not $O(n^2)$	$O(n)$
ChocolateDistribution	Sort + window	Width-m slide: $arr[i+m-1] - arr[i]$	$O(n \log n)$
LinkedListPalindrome	Linked list	getFirst/getLast + removeFirst/removeLast	$O(n)$
LinkedLists	LinkedList basics	addFirst loop + enhanced-for	$O(n)$
LongestSubstringWithoutRepeatingCharacters	Sliding window	HashSet shrink from left	$O(n)$
MatrixEvenOddSecondLargest	Partition + sort	Parity split; second largest = size() - 2	$O(n \log n)$

File	Concept	Key trick / API	Complexity
MaxFavouriteSong	Fixed window	Brute-force substring count (running count = $O(n)$)	$O(n \cdot k)$
PairSumwithMaximumProduct	Sort + two pointers	Converge on sum; track max product (pointer bug in Section 6)	$O(n \log n)$
RemoveAdj	Stack-like string	<code>delete(i, i+2) + Math.max(0, i-1)</code> rewind	$O(n^2)$ worst
SetMatrixZeros	Matrix marking	<code>boolean[]</code> row/col markers, two passes	$O(m \cdot n)$
StringDecoder	Run-length decode	<code>split("0"); (char)('A' + len - 1)</code>	$O(n)$
VowelPermutation	Recursion + counting	Consonant count $\rightarrow$ factorial via recursion	$O(c)$

Tier 3 - pointers, windows and simulations

SECTION

Section 8 - Revision Checklist

The compressed takeaway. Each line is one move that this corpus proves out; if you can write each one from memory in under a minute, every problem these files solve is within reach.

- **Count:** `map.put(k, map.getOrDefault(k, 0) + 1);` group with `computeIfAbsent`.
- **Sort and scan:** anagram (sort both), adjacent gap, width-m window, two-pointer pair search, second largest as `size() - 2`.
- **Walk two ends:** swap-reverse arrays; `removeFirst/removeLast` palindromes; converge pointers on a sorted target.
- **Window it:** variable window with a `HashSet`; fixed window with a running count.
- **Carry a prefix:** `rightSum = total - leftSum - arr[i];` adjacent scans bound at `length - 1`.
- **Play with digits:** `% 10` and `/ 10`; `i * i <= n` primes and divisor pairs; `% 15` LCM; Gauss sums; halving = `floor(log2)`.
- **Think in bits:** `Integer.bitCount`; `(x << 1) | 1` builds masks; XOR is a toggle state - never re-flip a prefix.
- **Build strings safely:** `StringBuilder` only; append-then-reverse; `delete(i, i+2) +` rewind for adjacent duplicates; `repeat(count)` for blocks.
- **Convert characters:** `charAt(i) - '0'` and `(char)('A' + k - 1)`; `indexOf` as membership.
- **Guard the edges first:** zero, one element, empty, null, negatives - then the loop.
- **Return exactly the contract:** the specified sentinel (-1 vs 0), the specified format, no prompts, no trailing whitespace - and prove it with the `stdout-capture` test.